

C to C++

Object-Oriented Programming in C++

- ▶ Implemented through the class construct
- ▶ Extension of struct in C
- ▶ We'll see examples illustrating OOP concepts:
 - ▶ Data hiding
 - ▶ Abstract data types (ADTs)
 - ▶ Inheritance and type hierarchies
 - ▶ Overloading and polymorphism
 - ▶ Templates

Output

```
// The first C++ program
#include <iostream>

int main() {
    std::cout << "C++ is an improved C.\n";
    return 0;
}
```

Explanation

- ▶ `//` denotes single-line comment
- ▶ `#include <iostream>` for I/O facilities
- ▶ `main()` In C++, the empty parentheses mean `main(void)`, never `main(...)`.
- ▶ `cout` for standard output
- ▶ `<<` to pass output string
- ▶ `endl` for newline and flush

Rewritten Program

```
#include <iostream>

main() {
    std::cout << "C++ is an improved C." << std::endl;
}
```

Rewritten Program

- ▶ the same output.
- ▶ drops the explicit declaration of `main()` return type; `int` is implicit.
- ▶ `<<` used twice, each time with `cout`, to continue from where printing was left off.
- ▶ `endl`, called a manipulator, forces a newline and flush.

Input

- ▶ In C++, variables need not be declared at the beginning of a block.
- ▶ Declarations may be mixed in with executable statements.
- ▶ Their scope extends from the point of declaration to the end of the block within which they are declared.

Input

```
#include <iostream>

int main() {
    const int moon = 238857;
    std::cout << "The moon's distance from Earth is " << moon
               << " miles." << std::endl;
    const int moon_kilo = moon * 1.609;
    std::cout << "In kilometers this is " << moon_kilo << " km."
               << std::endl;
    return 0;
}
```

Explanation:

- ▶ The keyword `const` creates a symbolic constant in C++, ensuring that the initialized value of `moon` cannot be changed.
- ▶ Stream I/O in C++ can handle various simple values without needing additional formatting information.

Input

```
#include <iostream>

const double m_to_k = 1.609;
inline int convert(int mi) { return (mi * m_to_k); }

int main() {
    int miles;
    do {
        std::cout << "Input distance in miles: ";
        std::cin >> miles;
        std::cout << "\nDistance is " << convert(miles)
                    << " km." << std::endl;
    } while (miles > 0);
    return 0;
}
```

Input

- ▶ The program repeatedly prompts the user for a distance in miles, terminating when a zero or negative value is entered.
- ▶ Constants reduce reliance on the preprocessor.
- ▶ The `inline` keyword specifies that a function is to be compiled, if possible, as inline code, avoiding function call overhead. It's better practice than `#define` macros.

Function Syntax

- ▶ Functions in C++ are like those in C.
- ▶ Types of parameters are listed inside `()`, enabling strong type checking and assignment-compatible conversions.
- ▶ Functions in C++ can have arguments directly called by reference, declared using the syntax: `type& identifier`.
- ▶ Function parameters can have default values, specified in the function declaration `()`-list as: `= expression`.

Example: add3 Program

```
#include <iostream>

inline void add3(int& sum, int a, int b, int c = 0)
    { sum = a + b + c; }

inline double average(int sum) { return sum / 3.0; }

int main() {
    int score_1, score_2, score_3, sum;
    std::cout << "Enter 3 scores: ";
    std::cin >> score_1 >> score_2 >> score_3;
    add3(sum, score_1, score_2, score_3);
    std::cout << "\nSum = " << sum;
    std::cout << "\nAverage = " << average(sum);
    std::cout << "\nWeighted Sum = " <<
        add3(sum, 2 * score_1, score_2);
    std::cout << std::endl;
    return 0;
}
```

add3

- ▶ The variable `sum` in `add3()` is call-by-reference, allowing direct manipulation within the function.
- ▶ Call-by-reference parameters must be lvalues, ensuring correct address usage during the procedure call.
- ▶ Default parameter values provide flexibility, allowing functions to be called with fewer arguments.

Classes in C++

- ▶ C++ class is an extension of the traditional C struct.
- ▶ Classes allow the implementation of user-defined data types along with associated functions and operators.
- ▶ Classes facilitate data abstraction, encapsulation, and code reusability.

OOP Concept vs. Equivalent in C

- ▶ **Class** - In C, equivalent to **Struct declaration**.
- ▶ **Object = Class instance** - In C, equivalent to **Struct instance**.
- ▶ **Instance method** - In C, equivalent to a **function that accepts pointers to Struct variable**.
- ▶ **Instantiation** - In C, equivalent to **allocation and reference through pointer**.
- ▶ **new keyword** - In C, equivalent to **calling malloc**.

Example: Implementation of a Basic String Class

```
#include <iostream>
#include <string.h>

const int max_len = 255;

class string {
private:
    char s[max_len];
    int len;
public:
    void assign(const char* st) { strcpy(s, st); len = strlen(st); }
    int length() { return len; }
    void print() { std::cout << s << "\nLength: " << len << "\n"; }
};
```


Explanation

- ▶ The class `string` implements a basic string data type with functions for assignment, length retrieval, and printing.
- ▶ The member functions `assign()`, `length()`, and `print()` provide universal access to the class data.
- ▶ The class encapsulates its data by declaring member variables `s` (character array) and `len` (length of the string) as private.

Usage of the string Class

```
int main() {  
    string one, two;  
    char three[] = "My name is Alan Turing.";  
  
    one.assign("My name is Charles Babbage.");  
    two.assign(three);  
  
    std::cout << three;  
    std::cout << "\nLength: " << strlen(three) << std::endl;  
  
    if (one.length() <= two.length())  
        one.print();  
    else  
        two.print();  
  
    return 0;  
}
```

Overloading

- ▶ In C++, overloading allows giving several meanings to operators or functions based on their context or arguments.

Function Overloading

- ▶ Function overloading allows defining multiple functions with the same name but different parameter lists.
- ▶ The appropriate function is selected at compile time based on the number and types of arguments passed to it.
- ▶ In C++, we can overload both member functions and non-member functions.
- ▶ Example: Overloading the 'print' function to accept an integer argument.

Example: Function Overloading

```
#include <iostream>
#include <string.h>

const int max_len = 255;

class string {
private:
    char s[max_len];
    int len;
public:
    void assign(const char* st) { strcpy(s, st); len = strlen(st); }
    int length() { return len; }
    void print() { std::cout << s << "\nLength: " << len << "\n"; }
    void print(int n) {
        for (int i = 0; i < n; ++i)
            std::cout << s << "\nLength: " << len << "\n";
    }
    friend string operator+(const string& a, const string& b);
};
```

Explanation

- ▶ The 'print' function in the 'string' class is overloaded to accept an integer argument.
- ▶ This version of 'print' prints the string 'n' times along with its length.
- ▶ Function overloading allows for flexibility and ease of use by providing multiple versions of the same function for different use cases.

Operator Overloading

- ▶ Operator overloading allows defining custom behaviors for operators when they are used with user-defined types.
- ▶ It enables us to use operators with custom types in a natural and intuitive way.
- ▶ Operators such as '+', '-', '*', '/' can be overloaded to work with user-defined types.
- ▶ Example: Overloading the '+' operator to concatenate two strings.
- ▶ The keyword `operator` precedes the operator token and together they behave like a function name in a function declaration.
- ▶ The keyword `friend` gives a function access to the private members of a class variable. A friend function is not a member of the class but has the privileges of a member function in the class in which it is declared.

Example: Operator Overloading

```
string operator+(const string& a, const string& b) {  
    string temp;  
    temp.assign(a.s);  
    temp.len = a.len + b.len;  
    if (temp.len < max_len)  
        strcat(temp.s, b.s);  
    else  
        cerr << "Max length exceeded in concatenation.\n";  
    return temp;  
}
```


Example: Operator Overloading

```
int main() {  
    string one, two, both;  
    char three[] = "My name is Alan Turing.";  
  
    one.assign("My name is Charles Babbage.");  
    two.assign(three);  
  
    if (one.length() <= two.length()) {  
        one.print();  
    } else {  
        two.print();  
    }  
  
    both = one + two;  
    both.print();  
  
    return 0;  
}
```

Explanation

- ▶ The '+' operator is overloaded as a friend function of the 'string' class to concatenate two strings.
- ▶ It takes two 'string' objects as arguments and returns a new 'string' object containing the concatenated result.
- ▶ Operator overloading allows us to use the '+' operator with 'string' objects just like built-in types.

Constructors and Destructors

- ▶ A constructor is a member function responsible for initializing an object of its class.
- ▶ Constructors are invoked whenever an object of their associated class is created.
- ▶ Constructors can be overloaded to support different initialization styles and parameter types.
- ▶ Example: Constructor for initializing a string with a pointer to a character value.
- ▶ A destructor is a member function responsible for deallocating or finalizing an object of its class.
- ▶ Destructors are called implicitly when an automatic object goes out of scope.
- ▶ Destructors are essential for cleaning up resources allocated by constructors, such as dynamic memory.
- ▶ Example: Destructor for deallocating memory allocated by a constructor.

Example: Constructor and Destructor for String

```
class string {  
private:  
    char s[max_len];  
    int len;  
public:  
    string(const char* st) { len = strlen(st);  
                           s = new char[strlen(st) + 1]; strcpy(s, st); }  
    string() { len = 255; s = new char[255]; }  
    string(int n) {len = n; s = new char[n + 1]; }  
    void assign(const char* st) { strcpy(s, st); len = strlen(st); }  
    int length() { return len; }  
    void print() { std::cout << s << "\nLength: " << len << "\n"; }  
    void print(int n) {  
        for (int i = 0; i < n; ++i)  
            std::cout << s << "\nLength: " << len << "\n"; }  
    friend string operator+(const string& a, const string& b);  
    ~string() { delete[] s; }  
};
```

Example: Constructor and Destructor for String

- ▶ All three constructors would be invoked in the following declaration: `string a, b(10), c("Sky is blue.");`
- ▶ Member initializer lists are used in C++ constructors to initialize member variables of a class. They appear in the constructor definition before the constructor's body and are a comma-separated list of initializations for each member variable.
- ▶ `ConstructorName(parameters) : member1(value1), member2(value2), ... {}`
- ▶ `circle(double r) : radius(r) {}`